

Moving large files across an air gap with a monitor and a camera

A technical overview of the Optical Transport Platform: what it does, how fast it goes, and what governs both.

Measured on commodity hardware — a 20-core workstation and an integrated 1080p webcam. Every figure in this document comes from a test in the repository, not an estimate.

Summary for management

Question	Answer
Does it work?	Yes, and it is verified rather than asserted. Every transfer is checked against a hash computed before anything was drawn, and the receiving endpoint recomputes it independently. Tested eleven ways end to end, including one frame in three deliberately destroyed.
How fast?	1.45 MB/s measured on one monitor and one camera — 8 MB of incompressible data in 5.5 seconds, zero retransmissions. Roughly 1 GB in 12 minutes.
Can it go faster?	Yes, and the route is horizontal: several screen–camera pairs carrying different chunks of the same file. Three pairs $\approx$ 6.6 MB/s. The design already supports it; the work is small (§8).
Can data be lost?	No. A chunk is shown repeatedly until the far side confirms it arrived intact. There is no path by which the sender moves on from an unconfirmed chunk.
What hardware?	A 4K monitor, a 1080p webcam, and a machine with cores to spare. No specialist equipment; both sides ran in containers on ordinary hardware for every figure in this document.
Biggest limitation?	Decoding cost — about 100 KB/s per CPU core, whatever the settings. The receiving side's core count sets the ceiling, which is why scale-out is the answer rather than tuning.
Not yet built?	Authentication on both APIs, and the multi-pair scale-out of §8. Both are named plainly here rather than left to be discovered.

The one idea to take away. Light only travels one way, and a frame lost to a flicker or a hand across the lens produces no error — only silence. So the sender never waits for bad news. It waits for good news, over a separate channel, and does not move on without it. Everything else in the system is an optimisation on top of that one rule.

1 · What the system is

Two computers with no network between them. One displays a file as a sequence of encoded images; the other photographs them and reassembles the file, then verifies it against a cryptographic hash the sending side declared before anything was drawn.

The only channel between them that is not optical carries acknowledgements — small signed records saying "chunk 47 arrived intact". That return path is what turns a hopeful broadcast into a lossless transfer, and it is the single most important idea in the design.

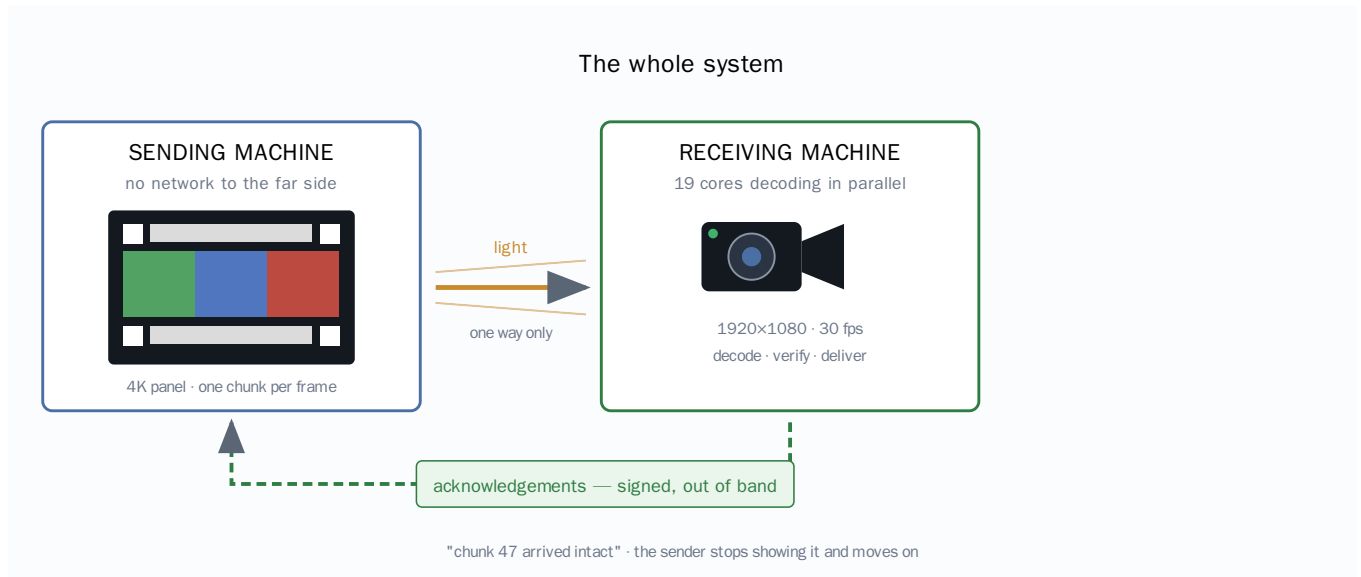


Figure 1. Data crosses as light and cannot be acknowledged optically — a lost frame produces no report, only silence. So the return path carries the acknowledgements, and the sender never advances past a chunk it has not heard about.

The guarantee, in one sentence. A chunk is displayed repeatedly until a signed acknowledgement for it arrives. Nothing else in the system promises delivery; error correction is an optimisation on top of that, not the guarantee.

2 · Speed: what the sender can offer

Throughput is bytes per frame × frames per second. Bytes per frame is set by how many cells the grid has and how many bits each cell carries; frames per second by the panel. Both are operator-controlled from the Settings page.

Measured payload capacity, from the encoder itself:

Grid	Frame on screen at 8 px cells	Bytes per frame color8 · 3 bits	Bytes per frame color16 · 4 bits	At 25 fps color16	At 60 fps color16
96×96	800 px	1 903	2 538	62 KB/s	149 KB/s
128×128	1 056 px	4 536	6 048	148 KB/s	355 KB/s

192×192	1 568 px	12 183	16 245	397 KB/s	953 KB/s
256×256	2 080 px	22 669	30 226	738 KB/s	1.73 MB/s
384×384	3 104 px (1 940 at 5 px)	52 716	70 288	1.68 MB/s	4.02 MB/s
512×512	4 128 px (2 064 at 4 px)	94 860	126 480	3.02 MB/s	7.24 MB/s

Capacity depends on the grid, not the cell size. A 256×256 grid carries the same 30 226 bytes whether its cells are 4 or 8 pixels. Cell size buys one thing only: the camera's ability to resolve a cell. It costs screen area and nothing else.

Refresh rate is a ceiling, not a target

A 144 Hz panel can show 144 frames a second. Setting the sender to 144 fps does *not* transfer 144 frames' worth of data, because the receiver cannot read that fast — and a frame nobody reads costs a render and delivers nothing. The panel's rate is measured in the browser on the Settings page (`requestAnimationFrame` intervals; 144 Hz shows as 6.94 ms) and offered as an upper bound.

Panel	Max fps	Ceiling at 384×384 color16	In practice
60 Hz office monitor	60	4.02 MB/s	Receiver-bound long before this
144 Hz gaming panel	144	9.65 MB/s	Receiver-bound long before this

Which is the point of the next section: on this hardware the sender is never the constraint.

3 · Speed: what the receiver can absorb

Decoding a frame is the most expensive thing the system does — locating four fiducials, fitting a homography, sampling every cell, demodulating, then checking a CRC32 and a SHA-256. Measured on one core:

Geometry	Per frame clean	Per frame through optics	Frames/s per core	Throughput per core
96×96 @ 6 px	21.8 ms	22.3 ms	44.9	86 KB/s
128×128 @ 8 px	55.6 ms	67.2 ms	14.9	67 KB/s
256×256 @ 8 px	236.8 ms	265.7 ms	3.8	85–115 KB/s

The striking result is that throughput per core is roughly constant — 85 to 115 KB/s — whatever the geometry. Bigger frames take proportionally longer. So the receiving side's capacity is essentially $cores \times 100\text{ KB/s}$, and the frame rate that makes sense follows from it:

frames per second = decode workers × 115 000 ÷ bytes per frame

Frames are decoded concurrently, one worker per core less one. Everything after the decode stays strictly sequential — chunk records, acknowledgements, the final merge — so no locking was introduced and the ordering guarantees are unchanged.

Cores for decoding	Sustainable rate	Frame rate to set at 384×384 color16	Time for 1 GB
1	115 KB/s	2 fps	2.4 hours
4	460 KB/s	7 fps	36 minutes
8	920 KB/s	13 fps	18 minutes
19 (this workstation)	2.2 MB/s	25 fps	7.6 minutes

Measured end to end

8 MB of deliberately incompressible data, both sides in containers, verified against the sender's hash and delivered to a callback endpoint that recomputed the hash independently:

Configuration	Offered	Achieved	Wall clock	Losses
256×256 @ 8 px, color16, 40 fps	1.15 MB/s	730 KiB/s	11.2 s	none
384×384 @ 5 px, color16, 25 fps	1.68 MB/s	1.45 MB/s	5.5 s	none

Prefer fewer, larger frames. Twice the throughput at less than two thirds the frame rate. Every frame costs a fixed overhead — a file read, an image decode, a database row, an acknowledgement —

regardless of how much payload it carries, so larger frames amortise all of it. This was not the expected result and is the most useful tuning lesson the project produced.

4 · What a chunk actually is

A file does not cross as a file. It is compressed, cut into chunks sized so that exactly one chunk fills exactly one frame, given error-correction shards their own frames, and rendered.

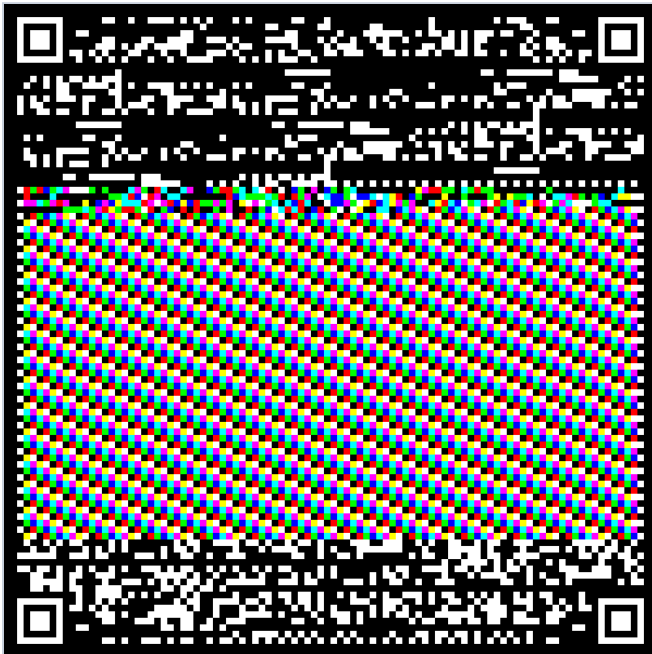


Figure 2. A real chunk frame from a transfer: 96×96 cells at 6 px, color8, three bits per cell, 1 903 payload bytes.

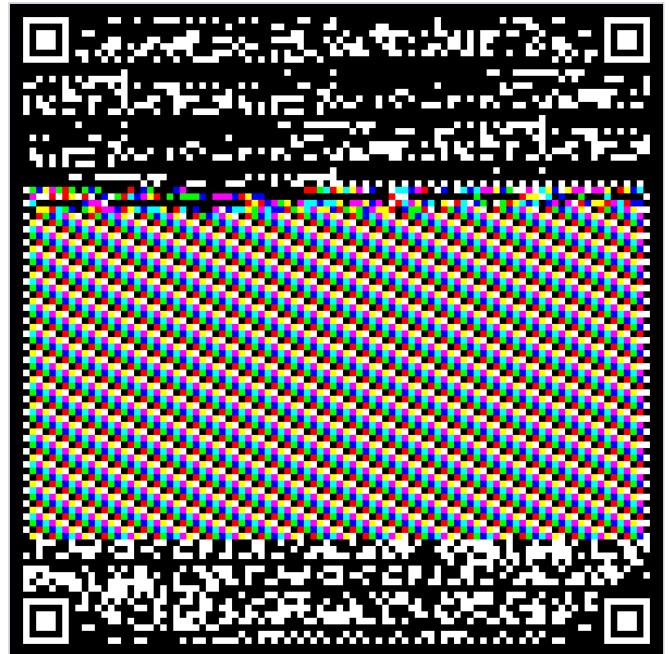
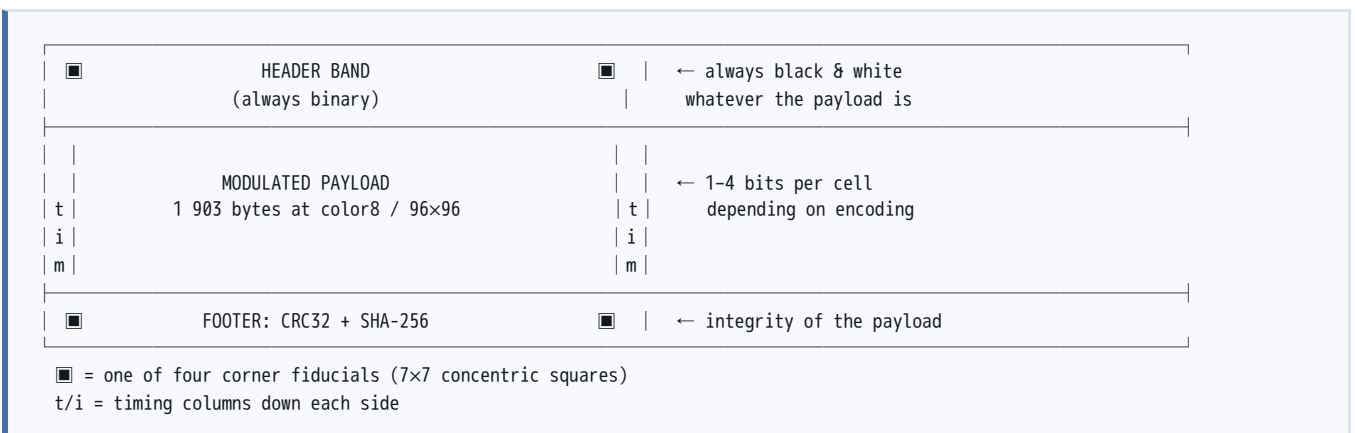


Figure 3. The manifest frame from the same transfer. Same geometry, different content — this is the frame that tells the receiver what everything else means.

The anatomy of one frame



Four design decisions are worth drawing out, because each one earns its cost:

- Four corner fiducials, not three. QR codes use three, which fixes rotation and scale. Four supply the correspondences for a full *homography* — so a camera mounted genuinely off-axis is corrected rather than merely tolerated.

- The header and footer are always plain black and white, whatever modulation the payload uses. This is what lets a receiver read a frame it knows nothing about: it learns the geometry and the encoding *from the frame*, and only then demodulates the payload.
- The footer carries both a CRC32 and a SHA-256 of the payload. The CRC is cheap and catches ordinary corruption; the hash makes a frame that satisfies it essentially certain to be intact.
- Timing columns down each side let the decoder track cell boundaries across the frame rather than trusting a single global grid estimate.

The header, field by field

Every frame is self-describing. This is the header a receiver reads before it knows anything else:

Field	Size	What it is for
Version	2 B	Protocol version; a receiver refuses what it cannot parse
Flags	2 B	Manifest / parity / encrypted markers
EncoderID , BitDepth	2 B	Which of the five encodings this frame uses
CompressionID , FECID	2 B	Which compressor and error-correcting code produced the payload
CellPixels , GridWidth , GridHeight	6 B	The geometry, so the frame explains its own shape
TransmissionID	16 B	Which file this belongs to
SessionID	16 B	Which display run produced it
FrameNumber , TotalFrames	8 B	Position in the display sequence
ChunkNumber , TotalChunks	8 B	Which piece of the file — the number that gets acknowledged
PayloadLength	4 B	How many payload bytes are real (the last chunk is short)
TimestampMS	8 B	When it was rendered

5 · How the settings control what is generated

Five independent choices, all on the sender's Settings page, and each changes the bytes that end up on screen. The header records every one of them, so the receiver never has to be told separately.

Setting	Options	What it changes	When it may change
Encoding	binary, grayscale, color8, color16, rolling	Bits per cell: 1 → 4. Directly multiplies capacity	Only between transfers
Grid	96×96 … 512×512	Cells per frame, so bytes per frame	Only between transfers
Cell size	4 – 8 px	Screen area and the camera's blur budget. <i>Not</i> capacity	Only between transfers
Compression	none, gzip, lz4, zstd, brotli	How much file becomes how many chunks	Per transfer
Error correction	none, Reed–Solomon, RaptorQ, LDPC	How many extra parity frames are shown	Per transfer
Frame rate	1 – 480 fps	How fast frames are shown. Nothing about their content	Any time, mid-transfer

Why geometry cannot change mid-transfer, and frame rate can. The grid, cell size and encoding are written into *every frame header*, and the chunk size was derived from the encoder's capacity at that grid. Changing them halfway would render the remaining chunks to a shape the manifest never declared, and the receiver would reassemble the wrong file. The system refuses it while anything is in flight and says so.

The frame rate is only a delay between two writes. Nothing already rendered depends on it, so it applies on the next frame — which is exactly when an operator wants it, because a receiver falling behind is the reason to turn it down.

How a change of algorithm changes the code on screen

The same 1 903 bytes of payload, drawn five different ways. The choice is visible at a glance:

Encoding	Bits per cell	What a cell looks like	Bytes per frame 96×96 grid	Tolerance
binary	1	Black or white	634	Survives the worst channel tested
grayscale	2–3	4 or 8 grey levels	1 269	Needs a controlled installation at 3 bits
color8	3	8 fully saturated hues	1 903	Survives the worst channel tested
color16	4	4 grey levels × 12 hues	2 538	Needs good optics or no optics at

rolling	varies	Bands, interleaved, each with its own CRC-16	varies	For rolling-shutter cameras
---------	--------	--	--------	-----------------------------

Two of these are worth a sentence of explanation for a non-specialist audience:

- `color8` is the workhorse. Eight fully saturated colours are far apart in colour space, so a lens that blurs and a sensor that adds noise still leave them distinguishable. It carries three times what black-and-white does at no cost in reliability.
- `color16` is the fast one and the fragile one. Sixteen levels means telling apart four brightnesses of twelve hues, which needs either a controlled installation or — as in the HTTP transport, where frames are fetched rather than photographed — no optics at all.

6 · How the receiver is told which algorithm was used

Two mechanisms, at different scales, and the distinction matters.

Per frame: the header

Every frame states its own encoding, bit depth, geometry, compression and error-correction identifiers, in a band that is *always plain black and white*. So a receiver that has just been switched on, knowing nothing, can read any frame it happens to catch: read the binary header, learn how the rest of this image is modulated, then demodulate it. Nothing has to be configured on the receiving side to match the sender.

Per transfer: the manifest frame

A header describes one frame. It cannot say how big the whole file is, what it is called, or what it should hash to. That is the manifest — a frame of its own, marked with a flag, sent first and then re-sent periodically so that a receiver joining late can still catch up.

Manifest field	What the receiver does with it
Filename	Names the reassembled file. Validated as a bare name, because it crosses from outside
OriginalSize , CompressedSize	Bounds decompression — every codec here can expand a small input without limit
SHA256	The verdict. Computed over the original file before anything was drawn
ChunkCount , ChunkSize	Knows how many pieces to expect and when it has them all
CompressionID	Which codec to decompress the merged bytes with
FEC (id, data shards, parity shards, shard size)	How to rebuild a chunk that never arrived from the parity ones that did
CallbackURL	Where to deliver the finished file — checked against an operator-held allowlist

Why the callback URL travels across the gap. The sending side is where a caller supplies it, and the receiving side is the one that ends up holding a finished file. It arrives from outside the receiver's trust boundary, so delivery is gated by an allowlist the receiving operator controls, and redirects are not followed — a redirect would be the far side choosing a second destination after the first had been approved.

7 · What happens to a captured frame

Nine steps, in order. The order is the design: each one exists to stop a specific failure being mistaken for a different one.

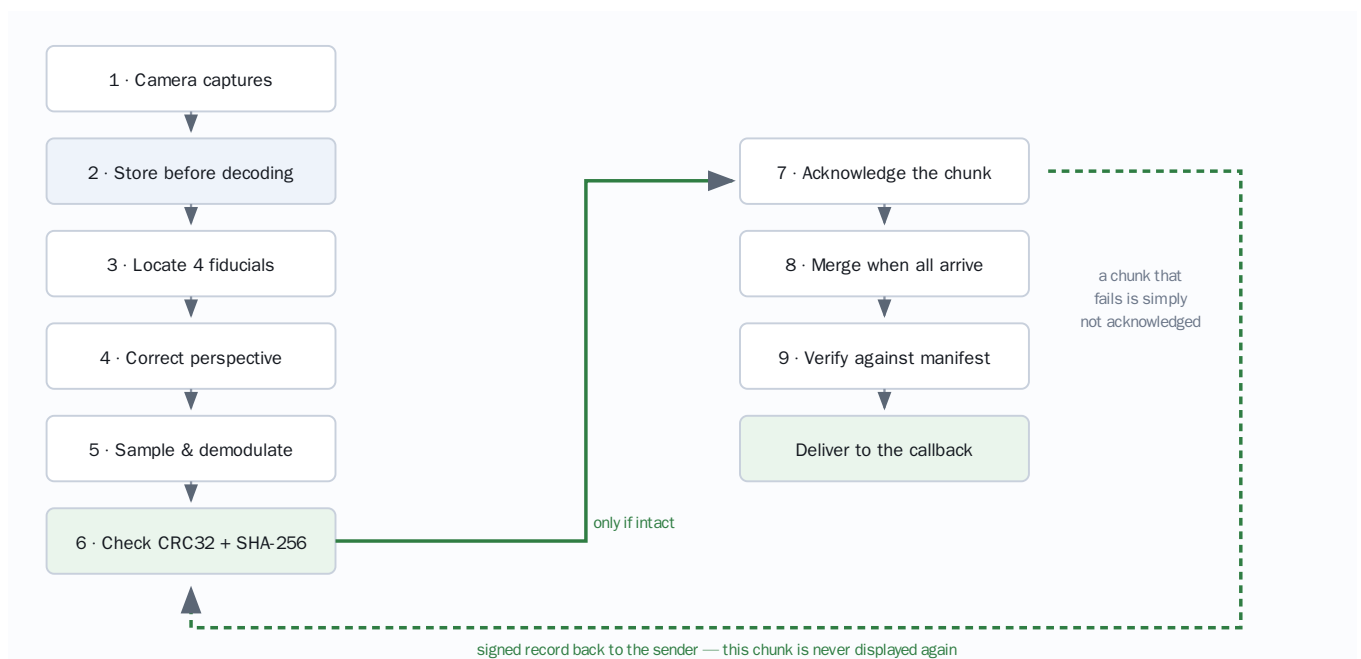


Figure 4. The receiving pipeline. Steps 3–5 run on 19 cores in parallel; everything after the decode is strictly sequential, which is why no locking was needed.

Why "store before decoding"

Every captured image is written to disk *before* anything tries to read it. A frame that cannot be decoded is the primary evidence for why a camera is going badly — blur, a tear, a hand across the lens — and it is the only thing a replay can work from. Discarding it would throw away the diagnosis along with the frame.

Why a failed chunk gets no reply at all

This is the part that most often surprises people, and it is deliberate. A frame that fails its checksum is not reported. There is no negative acknowledgement.

A frame that could not be read cannot say which chunk it was carrying — that information was in the part that failed to decode. So there is nothing to send a complaint *about*. The sender learns of the failure by its acknowledgement timing out, and shows the chunk again. Waiting for good news is the only mechanism that can work here; waiting for bad news would mean waiting for a message that can never be composed.

The acknowledgement, and why the same chunk stops appearing

When a chunk passes both checksums, the receiver writes a small signed record:

```
{
  "transmission_id" : "1adc37ba-fb0d-4c1d-a80e-ad62e0250924",
  "chunk_number"    : 47,
  "status"          : "ok",
  "bit_error_rate"  : 0.0,
  "received_at"     : "2026-08-07T14:33:02.881Z"
}
```

+ HMAC-SHA256 over the exact bytes above

The sender reads it, marks chunk 47 acknowledged, and never displays it again. Three consequences follow, and together they are the whole delivery model:

- The display converges. As acknowledgements arrive, the set of frames still worth showing shrinks. The display spends its time on what has not arrived rather than cycling through what has.
- The display never idles. If everything in the current window is awaiting acknowledgement, the oldest outstanding frame is shown again — a camera pointed at a blank screen learns nothing.
- A duplicate is not acknowledged twice. The sender already stopped showing that chunk and cannot unknow it, so a second record carries no information. Suppressing them was worth 4× in wall clock: a repeated five-transfer test went from 14.5 s to 3.4 s, and its rate stopped collapsing from 192 KB/s to 24 KB/s across runs.

The acknowledgement channel is the one input the sender takes from outside itself. Every record is signed with HMAC-SHA256 over its exact bytes, using a secret both sides hold and which has no default. Anything able to write that channel could otherwise report chunks as delivered when they were not — and the sender would move on, producing a file that is quietly incomplete.

8 · Going faster: many screens, many cameras, one database

A single monitor–camera pair is limited by the receiving machine's cores: roughly 100 KB/s per core, or 2.2 MB/s on the workstation measured here. The way past that is not a better camera. It is more pairs, running at once, carrying different chunks of the same file.

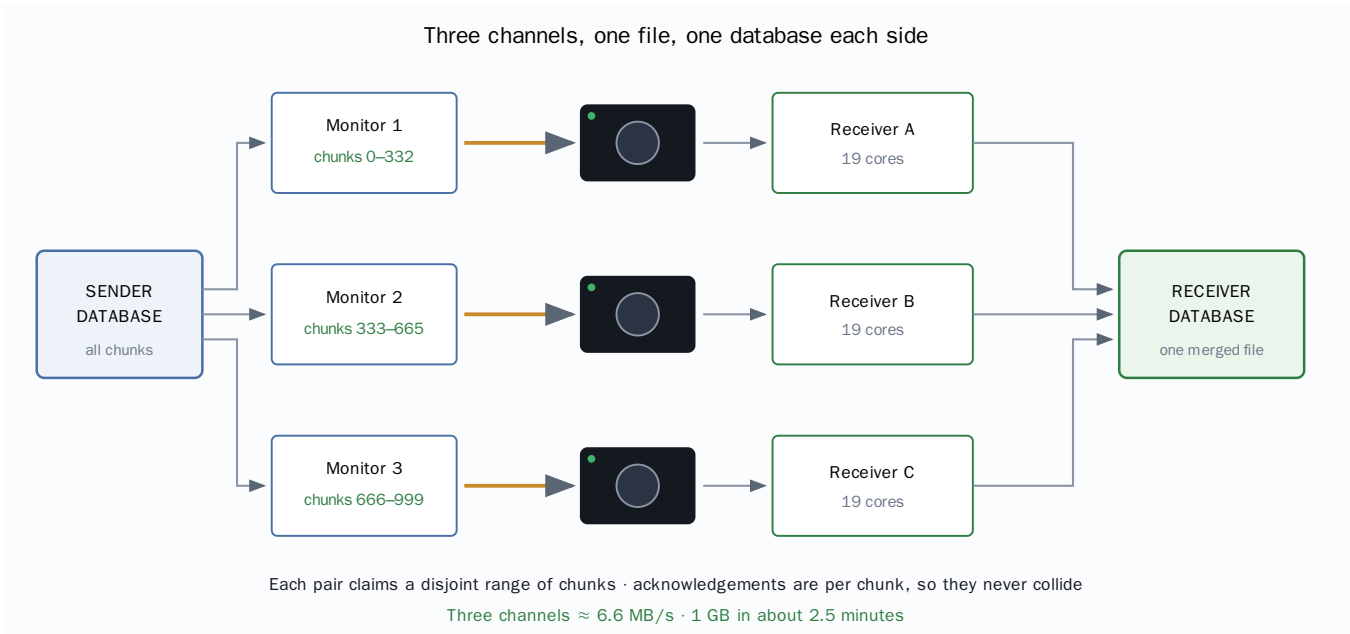


Figure 5. Horizontal scale-out. The database each side already holds one row per chunk, which is what makes disjoint claiming possible without inventing a new coordination mechanism.

Why the existing design already supports this

Three properties, all present today, are what make scale-out a matter of configuration rather than redesign:

- A chunk is addressed by number, not by position in a stream. Chunk 47 is chunk 47 whichever screen showed it and whichever camera read it. Nothing about reassembly depends on the order frames arrive in.
- The database already claims work atomically. The job engine uses `SELECT ... FOR UPDATE SKIP LOCKED`, which is precisely the primitive needed: several display loops ask for the next unacknowledged chunk and each is handed a different one, with no lock contention and no double-showing.
- Acknowledgements are per chunk and idempotent. Two channels cannot conflict, because the record says which chunk arrived and says nothing about who showed it.

What would need building

This is a proposal, not a shipped feature, and the honest scope is small but not nil:

Work	Effort	Why it is needed
Chunk-range claiming in the display loop	Small	Each display asks the database for the next unacknowledged chunk rather than walking its own list
One receiver per camera, sharing a database	Small	The pipeline already writes chunk rows; several instances against one database need no change beyond configuration

Merge triggered by count, not by one process	Small	Whichever receiver records the last chunk performs the merge; the others see it is done
Per-channel session accounting	Moderate	So an operator can see which screen–camera pair is underperforming rather than one blended figure

What it buys. Throughput becomes linear in the number of pairs, because the pairs share nothing but a database. Three pairs on the hardware measured here is roughly 6.6 MB/s — 1 GB in about two and a half minutes. Ten pairs, on ten inexpensive machines, would be about 22 MB/s: 1 GB in 45 seconds.

Where it stops being linear. The shared database becomes the coordination point, and at some number of pairs the acknowledgement write rate is the constraint rather than the cameras. On the figures measured here that point is a long way out — acknowledgements are a few hundred bytes and a few hundred per second per channel — but it is where the next ceiling lives, and it should be measured rather than assumed before anyone commits to a pair count.